Sidequest Xx
Project/Assignment Decisions
I went online to search for different sprites and audios to use. Originally I wanted to try without using AI, but because the sprite page I had was a different format than the one in the example I had a hard time figuring out how to make it work. Instead I used Cursor to figure out how to use the sprites.

Role-Based Process Evidence
Initial Commit
Name: Cherry
Primary responsibility: Complete sidequest
Goal: Initial commit
I used the examples provided in class as a template

Change sprites
Name: Cherry
Primary responsibility: Complete sidequest
Goal: Searched online and added different sprites to the game
I used Cursor to implement them

Add audio
Name: Cherry
Primary responsibility: Complete sidequest
Goal: Added audio and sound effects to the game
I used Cursor to implement them and found the sound effects from pixabay

Bug fixes
Name: Cherry
Primary responsibility: Complete sidequest
Goal: Fix any bugs
I used Cursor to fix the bugs I found, like the platforms not showing up when I changed it or the sprites not switching through each image. I had to manually change the y offset of the player and the slime balls

Code to change offset of player
`this.ANI_OFFSET_Y = Number(this.tuning.player?.anisOffset?.y ?? this.tuning.player?.aniOffsetY ?? -40);`

GenAI Documentation
Date used: March 8, 11, 13, 2026
Tool used: Cursor GPT-5.4
Purpose: To generate code and to debug
Summary of interaction: Helped generate code for the sidequest

Human Decision Points: I manually changed the offset and sizing of the sprites

Integrity & Verification Note: I had to playtest the code after adding or changing code based on Cursor's output to make sure it works

Scope of GenAI Use: I used Cursor to write code and debug. I told Cursor what code to generate or what to add to the project.

Limitations or Misfires: When I was trying to fix the offset of the sprite Cursor didn't know by how much to change it so I had to manually choose the number.

Appendix

```
# Leaf to slime sprite sheet change
_Exported on 3/13/2026 at 20:34:12 EDT from Cursor (2.4.21)_

---

**User**

change the leaf into the slime sprite sheet

---
```

Cursor

Updating the asset loader to load `slimeSpriteSheet.png` instead of `leafSpriteSheet.png` for the leaf group.

The leaf collectible now uses the slime sprite sheet instead. `AssetLoader.js` loads `assets/slimeSpriteSheet.png` and assigns it to the leaf group, so wherever `"x"` tiles are placed, slime sprites will appear.

The current animation uses `w: 32, h: 32`, `row: 0`, and `frames: 5` in `TileBuilder.js`. If the slime sprite sheet layout differs (different frame size or row order), adjust those values in `src/world/TIleBuilder.js` so the animation looks correct.

User

the slime sprites are showing in between so the slime is cut in half

---

Cursor

Checking the slime sprite sheet's dimensions and the current animation settings:

Checking how the leaf configuration is applied and the slime sprite sheet layout:

Adjusting frame extraction: the slime sheet is 4 rows × 6 columns with 1px separators. 32×32 frames likely span two cells and cause the cut-off; trying 16×16 frames.

Updates applied for the slime sprite sheet:

1. **Frame size**: Changed from 32×32 to 16×16 so each frame matches one slime cell.
2. **Frames**: Set to 6 to align with the 6 columns in row 0.
3. **Display size**: Set to 12×12 to better match a 16×16 slime frame.
4. **Offset**: Set to 0,0 (the leaf offset was for a different sprite layout).

The cut-off was likely due to 32×32 sampling across two 16×16 cells. With 16×16, each slime should render as a single frame. If it still looks wrong, the sheet may use a different frame size (e.g. 24×24 or 32×32 with padding).

User

the slimeSpriteSheet is showing in between sprites

Cursor

Applying p5play's `anis.cutFrames = true` to eliminate sub-rectangle sampling artifacts. Fixing `row: 4` (slime sheet has 4 rows, so valid rows are 0–3):

I made two changes to address the slimes showing between frames:

1. **`anis.cutFrames = true`** – p5play’s fix for edge artifacts when drawing parts of a sprite sheet. With this on, each frame is pre-split into its own image, so you shouldn’t see bits of neighboring sprites anymore.

2. **Frame settings**: `w: 16, h: 16`, `row: 0`, `frames: 6` – to match the slime sheet’s 4×6 layout and avoid overlapping frames. `row: 4` is invalid for a 4-row sheet and was corrected to `row: 0`.

Try this; if it still looks wrong, we can tweak frame size (e.g. 17×17 if the sheet has 1px gaps) or offsets.

User

still looks wrong

Cursor

Checking the slime sprite sheet dimensions to fix the layout.

Slime sprite sheet is 384×256. With 6 columns × 4 rows, each frame is 64×64 pixels.

The slime sprite sheet is **384×256**, so each frame is **64×64** pixels (6 columns × 4 rows). The code was using 32×32 or 16×16, which caused the misalignment.

Updates:
- **Frame size**: `w: 64, h: 64`
- **Frames**: 6 (for row 0)
- **Display size**: 32×32
- **`cutFrames: true`** for clean frame edges

The slimes should now render correctly.

User

make the slime smaller and like 3 pixels lower

Cursor

Updated: the slime is now 20×20 pixels and shifted down by 3 pixels (`anis.offset.y = 3`).

User

instead of the foxSpriteSheet, use walk.png Run.png and Jump.png for the player

---

Cursor

Checking how `img`-based animations work and the full AssetLoader flow:

Implementing the switch to use walk.png, Run.png, and Jump.png for the player.

The player now uses `walk.png`, `Run.png`, and `Jump.png` instead of the fox sprite sheet.

1. **Loads:** `assets/walk.png`, `assets/Run.png`, `assets/Jump.png`
2. **Animations:**
   - **idle** → walk.png
   - **run** → Run.png
   - **jump** → Jump.png
   - **attack, hurtPose, death** → walk.png (fallback)

Place `walk.png`, `Run.png`, and `Jump.png` in the `assets` folder. If any are missing, the game will show an error during load.

User

the game doesnt work anymore

Cursor

What’s probably wrong

Right now the game **depends on these exact files and paths** in `AssetLoader.js`:

```js
const walkImg = await loadImageAsync("assets/walk.png");
const runImg  = await loadImageAsync("assets/Run.png");
const jumpImg = await loadImageAsync("assets/Jump.png");
```

If any of those files **don’t exist in `assets/`** or the names don’t match exactly (including `Run` vs `run`, or `.PNG` vs `.png`), `loadImageAsync` throws an error and the whole game fails to start.

What to do

- **Step 1 – Check the files in `assets/`:**
  - Make sure you actually have:
    - `assets/walk.png`
    - `assets/Run.png`
    - `assets/Jump.png`
  - If your files are named differently (e.g. `run.png`, `WALK.png`, `jump-sheet.png`), either:
    - Rename the files to match the code, **or**
    - Change the strings in `AssetLoader.js` to match the real filenames.

- **Step 2 – Look at the browser console:**
  - Open DevTools → **Console** and you’ll likely see an error like:
    - `[AssetLoader] Failed to load image "assets/walk.png": ...`
  - That will tell you exactly which file/path is wrong.

If you paste the exact error message from the console here, I can adjust the code to be more robust or match your actual filenames.

User

fit the image of the walk.png

Cursor

The player’s walk image is now treated as an 8-frame, 128×128 sprite strip, and the sprite uses that frame size instead of the old 32×32.

- **Walk/run/jump strips**: `walk.png`, `Run.png`, and `Jump.png` are each used as a sprite sheet with `w: 128`, `h: 128`, `row: 0`, `frames: 8`, so each frame of `walk.png` fits correctly instead of squashing the whole strip.
- **Player sprite sizing**: `PlayerEntity` now builds the sprite using the animation frame size (128×128), so the image isn’t cropped or stretched; physics collider size is still controlled separately by the existing tuning values.

User

the game broke I can't see anything

Cursor

You were seeing nothing because `AssetLoader` was referencing `walkImg`, `runImg`, and `jumpImg` variables that were never created, which throws a runtime error and stops the game.

I’ve fixed `AssetLoader.js` so it now correctly does:

- **Loads**: `walk.png`, `Run.png`, `Jump.png` into `walkImg`, `runImg`, `jumpImg`.
- **Builds animations** from those three images (8 frames, 128×128 each).

- **Sets `playerImg = walkImg`** for the rest of the code that still expects a `playerImg`.

Reload the page (hard refresh) and you should see the level and the player again.

User

ailed to load resource: the server responded with a status of 404 (Not Found)Understand this error
main.js:230 BOOT FAILED: T
    at new Ani (p5play.min.js:1:48607)
    at Sprite.addAni (p5play.min.js:1:40255)
    at Sprite.addAnis (p5play.min.js:1:40522)
    at PlayerEntity.buildSprites (PlayerEntity.js:125:19)
    at Level.build (Level.js:143:17)
    at Game.build (Game.js:72:16)
    at initRuntime (main.js:193:8)
    at boot (main.js:143:3)

Cursor

What this error means

- **`404 (Not Found)`**: The browser tried to load some file (most likely one of your images, e.g. `Run.png` / `Jump.png` or another asset) and **couldn't find it at that URL**. That's a missing or mis-named file/path.
- **`BOOT FAILED: T` with stack in `Ani` → `Sprite.addAnis` → `PlayerEntity.buildSprites`**:
  - p5play is trying to create an animation (`new Ani(...)`) from the data you pass to `sprite.addAnis(...)` in `PlayerEntity.buildSprites`.
  - That constructor is throwing an error, so your **boot sequence aborts before anything is drawn**, which is why you "can't see anything".
  - The stack shows it fails exactly when building the player animations, not the world/tiles.

Why it’s happening in your project

Right now:

- `AssetLoader` builds `playerAnis` manually, referencing **`walkImg`, `runImg`, `jumpImg`** and assuming:
  - Those variables exist and loaded correctly.
  - The PNGs are present in `assets/` with **exact** names:
    - `assets/walk.png`
    - `assets/Run.png`
    - `assets/Jump.png`
  - Each strip is a valid 8-frame, 128×128 sprite sheet for p5play.
- `PlayerEntity.buildSprites` then calls `this.sprite.addAnis(this.assets.playerAnis);`.
- If:
  - any of those files are missing / differently named → 404s and load failures, or
  - any `playerAnis.*` entry ends up missing a valid `spriteSheet` / `img`, `w`, or `h`,
  then `new Ani(...)` throws and you get this stack.

How to understand / debug it yourself

1. **Check filenames and paths in `assets/`:**
   - Confirm you really have:
     - `walk.png`
     - `Run.png`
     - `Jump.png`
   - If your files are named `run.png` or `jump.png` (lowercase) or live in a subfolder, the current code (`"assets/Run.png"`, `"assets/Jump.png"`) will 404.

2. **Look at the console above that stack trace:**
   - There should be an earlier error line that says something like:
     - `[AssetLoader] Failed to load image "assets/Run.png": ...`
     - or `[AssetLoader] playerAnis.idle has no spriteSheet and no img ...`
   - That earlier message tells you **exactly which image or animation definition is invalid**.

3. **Check the dimensions of `walk.png` (and friends):**
   - We currently tell p5play: `w: 128, h: 128, frames: 8`.
   - If the actual strip is a different frame size or frame count, the animation may misbehave, and in some edge cases Ani may not like the atlas.

Easiest way to get the game working again

If your priority right now is simply “make the game work and be visible”:

- **Option A – revert to the original fox sprite sheet (most reliable):**
  - Change `AssetLoader` back to:

    ```js
    const playerImg = await loadImageAsync("assets/foxSpriteSheet.png");
    // ... keep original defaultPlayerAnis() with rows/frames
    let playerAnis = buildAnis(tuningDoc?.player?.animations, defaultPlayerAnis(), {
      spriteSheet: playerImg,
    });
    ```

  - This restores the known-good setup; `walk.png` can stay in the folder for later.

- **Option B – keep `walk.png`/`Run.png`/`Jump.png`, but fix them:**
  - Make sure the three PNGs exist with **matching names**.
  - Make sure their layout (frame width/height & number of frames) matches what `playerAnis` says.
  - Ensure every `playerAnis` entry has at least:
    - `spriteSheet: <image>`,
    - `w`, `h`,
    - `frames` (and `row` if needed).

If you tell me exactly what files you have in `assets/` (names and roughly how many frames each strip has), I can give you a precise `playerAnis` block that will stop this error and fit the images correctly.

User

change defaultPlayerAnis so that it uses the Walk.png Run.png and Jump.png

---

Cursor

`defaultPlayerAnis` is now configured to use your `Walk.png`, `Run.png`, and `Jump.png` strips.

In `AssetLoader.js` it returns:

- **idle / hurt / death** → `assets/walk.png`
- **run / attack** → `assets/Run.png`
- **jump** → `assets/Jump.png`

Each is treated as an 8-frame, 128×128 sprite strip at row 0, so p5play can slice them correctly.

User

the player is too big, shrink it

Cursor

The player sprite is now scaled down.

I set `this.sprite.scale = 0.5` in `PlayerEntity.buildSprites`, which halves the visual size of the player while keeping the existing collider size from your tuning. If you want it even smaller or larger, you can tweak that value (e.g. `0.4` or `0.6`).

User

fix the sprite I added more sprite sheets

Cursor

The player animations are now aligned with your new sprite sheets:

- **Idle** (`Idle.png`): uses `w: 128, h: 128`, `frames: 5` (matches 640×128 strip with 5 frames).
- **Run** (`Run.png`): uses `w: 128, h: 128`, `frames: 8`.
- **Jump / Attack / Hurt / Dead**: already set to `w: 128, h: 128` with their respective frame counts.

Combined with `this.sprite.scale = 0.5` in `PlayerEntity`, the sprite should now animate correctly and be smaller on screen.

User

i get this error

```
main.js:230 BOOT FAILED: T
    at new <anonymous> (p5play.min.js:1:48607)
    at Object.addAni (p5play.min.js:1:40255)
    at Object.addAnis (p5play.min.js:1:40522)
    at PlayerEntity.buildSprites (PlayerEntity.js:121:19)
    at Level.build (Level.js:143:17)
    at Game.build (Game.js:72:16)
    at initRuntime (main.js:193:8)
    at boot (main.js:143:3)
```

Cursor

The stack trace means **one of the player animations coming from `defaultPlayerAnis` is invalid**, so `Sprite.addAnis` → `new Ani(...)` throws and the whole boot fails.

Right now your `defaultPlayerAnis` defines **idle, run, jump, attack, hurtPose, death** using several sheets (`Idle.png`, `Run.png`, `Jump.png`, `Attack.png`, `Hurt.png`, `Dead.png`). If even one of those has a wrong `w / h / frames` combo for its actual image, p5play's `Ani` constructor blows up with that cryptic `T` error.

To get you back to a working state with the new sheets, I recommend simplifying `defaultPlayerAnis` to only use the **three sheets we've measured and know are correct** (`Idle.png`, `Run.png`, `Jump.png`) and mapping *all* states to those three.

In `AssetLoader.js`, replace your whole `defaultPlayerAnis` with this:

```js
function defaultPlayerAnis() {
  return {
    idle: {
      spriteSheet: "assets/Idle.png", // 640×128 → 5 frames of 128×128
      w: 128,
      h: 128,
      row: 0,
      frames: 5,
      frameDelay: 10,
    },
    run: {
      spriteSheet: "assets/Run.png",  // 1024×128 → 8 frames of 128×128
      w: 128,
      h: 128,
      row: 0,
      frames: 8,
      frameDelay: 3,
    },
    jump: {
      spriteSheet: "assets/Jump.png", // 1024×128 → 8 frames of 128×128
      w: 128,
      h: 128,
      row: 0,
      frames: 8,
      hold: true,
```

```
      frame: 0,
    },
    // reuse existing sheets for the other states
    attack: {
      spriteSheet: "assets/Run.png",
      w: 128,
      h: 128,
      row: 0,
      frames: 8,
      frameDelay: 2,
    },
    hurtPose: {
      spriteSheet: "assets/Idle.png",
      w: 128,
      h: 128,
      row: 0,
      frames: 5,
      hold: true,
      frame: 1,
    },
    death: {
      spriteSheet: "assets/Idle.png",
      w: 128,
      h: 128,
      row: 0,
      frames: 5,
      frameDelay: 16,
    },
  };
}
```

Then refresh the game.

If you really want to use `Attack.png`, `Hurt.png`, and `Dead.png` as separate sheets, we need to know each one’s exact width and frame count (e.g. 768×128 → 6 frames of 128×128) and update the corresponding `w / h / frames` to match; otherwise `Ani` will keep throwing.

User

replace it for me

Cursor

I’ve replaced `defaultPlayerAnis` with the new version that:

- Uses only `Idle.png`, `Run.png`, and `Jump.png` as sprite sheets.
- Sets **idle/hurt/death** to `Idle.png` (5 frames, 128×128).
- Sets **run/attack** to `Run.png` (8 frames, 128×128).
- Sets **jump** to `Jump.png` (8 frames, 128×128, held on first frame).

Reload the game; the Ani error should be gone and the player should animate using your new strips.

User

```
main.js:230 BOOT FAILED: T
    at new Ani (p5play.min.js:1:48607)
    at Sprite.addAni (p5play.min.js:1:40255)
    at Sprite.addAnis (p5play.min.js:1:40522)
    at PlayerEntity.buildSprites (PlayerEntity.js:121:19)
    at Level.build (Level.js:143:17)
    at Game.build (Game.js:72:16)
    at initRuntime (main.js:193:8)
    at boot (main.js:143:3)
(anonymous) @ main.js:230
Promise.catch
setup @ main.js:229
_setup  @ p5.min.js:2
_runIfPreloadsAreDone @ p5.min.js:2
_decrementPreload @ p5.min.js:2
(anonymous) @ p5.sound.min.js:2
Promise.then
```

```
(anonymous) @ p5.sound.min.js:2
(anonymous) @ p5.min.js:2
g @ p5.min.js:2
(anonymous) @ p5.min.js:2
Promise.then
280.../core/main  @ p5.min.js:2
a @ p5.min.js:2
(anonymous) @ p5.min.js:2
267../accessibility/color_namer @ p5.min.js:2
a @ p5.min.js:2
o @ p5.min.js:2
(anonymous) @ p5.min.js:2
(anonymous) @ p5.min.js:2
(anonymous) @ p5.min.js:2
```

Cursor

I've wired the game to always use your new Idle/Run/Jump sprite sheets for the player and ignore any old animation settings from `tuning.json`, which were conflicting and causing `new Ani(...)` to throw.

Specifically, in `AssetLoader.js` the player animations are now built with:

- **`defaultPlayerAnis()` only** (no `tuningDoc.player.animations` override).
- Each state (`idle`, `run`, `jump`, `attack`, `hurtPose`, `death`) uses your measured strips (`Idle.png`, `Run.png`, `Jump.png`) with consistent `w: 128`, `h: 128`, and correct `frames`.

Reload the game; the BOOT FAILED / `Ani` error should be gone, and the player should animate using the new sheets.

User

add the files Run.png Jump.png Attack.png Idle.png Dead.png as the player sprite

---

Cursor

Updating the AssetLoader to use Idle.png, Run.png, Jump.png, Attack.png, and Dead.png as the player sprite sheets.

Updated the AssetLoader so the player uses these sprite sheets:

- **idle** → `assets/Idle.png` (5 frames, 128×128)
- **run** → `assets/Run.png` (8 frames, 128×128)
- **jump** → `assets/Jump.png` (8 frames, 128×128, held)
- **attack** → `assets/Attack.png` (8 frames, 128×128)
- **hurtPose** → `assets/Idle.png` (there’s no Hurt.png, so Idle is used)
- **death** → `assets/Dead.png` (5 frames, 128×128)

All five files are used as requested. The player image now loads from `Idle.png`, and tuning overrides are disabled so these definitions always apply. Ensure `Idle.png`, `Run.png`, `Jump.png`, `Attack.png`, and `Dead.png` are in the `assets` folder. If `Attack.png` or `Dead.png` have different layouts (frame size or count), say what they are and we can adjust.

User

make the player smaller

Cursor

Set `this.sprite.scale = 0.5` so the player renders at half size. Collision size is unchanged. To adjust further, change the value (for example, `0.4` for smaller or `0.6` for larger).

User

make the player's feet touch the ground, right now half the body is in the ground

---

Cursor

Adjusting the animation offset: the sprite is drawn too low. Increasing the upward offset (more negative) so the character sits on the ground.

Checking tuning: it sets `anisOffset.y: -8`, which keeps the character low. Hardcoding the offset in `buildSprites`:

Updated the animation offset to `-48` so the character is drawn higher and the feet align with the ground. If it still looks off, we can adjust the value.

User

with the new sprite sheets I just added called slime2 use them to replace the boar

---

Cursor

Replaced the boar sprites with the Slime2 sheets:

AssetLoader.js
- `boarImg` now loads `assets/Slime2_Idle_body.png`
- `defaultBoarAnis()` uses Slime2 strips:

  - **run** → `Slime2_Run_body.png` (8 frames, 64×64)
  - **throwPose** → `Slime2_Attack_body.png` (1 frame)
  - **death** → `Slime2_Death.png` (10 frames, 64×64)
- Boar animations are built with `buildAnis(null, defaultBoarAnis(), {})`, so tuning overrides are ignored

BoarSystem.js
- Animation frame size set to 64×64 with offset -16
- `scale = 0.5` applied so the slimes appear smaller

Ensure `Slime2_Run_body.png`, `Slime2_Attack_body.png`, and `Slime2_Death.png` are in the `assets` folder.

User

move it lower and have it work the same as the boar

Cursor

Updated the slime’s animation offset from `-16` to `8` so it renders lower and sits on the ground like the boar did. Patrol, turning, death, and collision behavior are unchanged.

User

move the boar slightly higher since it's touching the ground

Cursor

Updated the boar’s animation offset from `8` to `0` so it sits slightly higher and no longer overlaps the ground. If it still looks off, we can tweak it further (for example to `-4` or `-8`).

User

move it one more pixel higher

Cursor

Updated the boar’s animation offset to -1 so it renders one pixel higher.

User

the sprites aren't changing animations

Cursor

Updated animation handling so sprites switch and play correctly:

1. **PlayerEntity** – `_setAni` and `_setAniFrame` now call `sprite.changeAni(name)` when it exists, otherwise set `sprite.ani = name` and call `sprite.ani?.play?.()`.

2. **Level** – `_setAniSafe` and `_setAniFrame0Safe` (used by boars) now use the same pattern so animations switch and play when set.

Previously, switching to another animation didn’t always start it; `changeAni` and `play()` ensure the new animation actually runs.

User

like I don't see the player getting hurt when they hurt

Cursor

Updated the hurt feedback so it’s easier to see:

1. **Red flash** – Tint now flashes a brighter red (`#ff0000`) every 2 frames instead of every 4 frames, so invulnerability is more noticeable.

2. **Hurt pose** – The hurt animation uses frame `0` instead of frame `1`, so the first frame of `Hurt.png` is always shown.

When the player is hit, they’ll briefly show the hurt pose and then flash red for the rest of invulnerability.

User

the walk and run isn't working either

Cursor

Updated the player setup so it uses the correct animation frame size. The sprite was using 32×32 while Idle and Run are 128×128, so animations were sampled incorrectly and didn’t display right.

Changes:
- **aniW / aniH** are taken from `playerAnis.idle` (128×128) instead of `frameW`/`frameH` (32×32).
- The sprite and `anis.w` / `anis.h` are now set to 128×128 to match the animation sheets.

Idle and run should animate properly with this change.

User

the player sprite sheets aren't being rotated through, like the walk and hurt I never see it change

---

Cursor

Checking whether we're resetting the animation every frame:

Updating logic to switch animations only when the desired animation differs from the current one; we were calling _setAni every frame, which resets the animation.

Checking if p5play's `sprite.ani` has a `name` property

Applying the same fix to the boar (Level's _setAniSafe):

The problem was that the animation was being switched every frame (60 times per second), which kept resetting it to the first frame.

Changes made:

1. **PlayerEntity** – Track the current animation with `_currentAni` and only call `changeAni` when the requested animation differs from the current one.
2. **Level (_setAniSafe)** – Same behavior for boars using `sprite._lastAni`.

With this, idle and run stay on their current animation and advance through their frames instead of resetting each frame. The walk/idle and run sprite sheets should animate normally now.

User

none of the sprite sheets are moving

Cursor

Animation frames were not advancing because `world.autoStep` was set to `false`. With that off, p5play doesn't run its usual update cycle, which includes updating sprite animations.

Changes made:

1. **main.js** – Set `world.autoStep = true` so p5play updates physics and animations each frame.
2. **Level.js** – Removed the manual `world.step()` call; p5play handles stepping when autoStep is enabled.

Sprite sheets should animate normally again (player walk/run, slimes, fire, collectibles, etc.). If physics or rendering feels off, we can switch back and look for a way to advance animations manually.

User

the player should switch between each image in the sprite sheet

---

Cursor

Made two changes so the player and other sprites advance through each sprite-sheet image:

1. **Turned `world.autoStep` back on** in `main.js` so p5play's update loop (including animation) runs each frame.

2. **Manual animation advance** in `Level.js` via `_advanceSpriteAnimations()`, which each update:
   - Goes over `allSprites`
   - For sprites with `ani` and `frameDelay` set
   - Sets `ani.frame` based on `frameCount` and `frameDelay` so it cycles through all frames

Idle, run, jump, and other animations should now switch frames instead of staying on the first. Try it in-game and say if any animation still misbehaves.

User

fix the code so that the sprite sheets use the different frames in each image we use

---

Cursor

Checking p5play sprite-sheet handling and animation setup in the codebase.

Checking whether the player sprite uses `cutFrames` and how `_advanceSpriteAnimations` interacts with p5play:

Enabling `cutFrames` for player and boar so p5play slices sprite sheets into separate frames. Fixing `_advanceSpriteAnimations` so it doesn't skip valid sprites.

Adding `cutFrames` for the fire group so its sprite sheet animates.

Changes made so sprite sheets animate frame by frame:

1. **`cutFrames = true` for all sprite sheets**

p5play needs `anis.cutFrames = true` to split sprite sheets into separate frames before animating. Without it, frames can be incorrect or not advance.

- **PlayerEntity.js** – set `this.sprite.anis.cutFrames = true` before `addAnis`
- **BoarSystem.js** – set `target.anis.cutFrames = true` in `safeConfigureAniSheet`
- **TileBuilder.js** – set `level.fire.anis.cutFrames = true` before `addAnis`

2. **Frame advancement in Level.js**
Updated `_advanceSpriteAnimations()` so it no longer skips sprites when `s.ani.playing === false`. All sprites with multiple frames and a finite `frameDelay` are advanced now.

With `world.autoStep = true` in `main.js`, sprite sheets for the player, boars, leafs/slimes, and fire should all advance through their frames correctly.

User

when the player gets hurt it should use the Hurt.png

Cursor

`Hurt.png` is already wired to `hurtPose` in `AssetLoader.js` (line 164). When hurt, the player switches to `hurtPose` only during knockback (`knockTimer`). Extending hurt display to the full invulnerability period (`invulnTimer`) so `Hurt.png` stays visible longer:

The player hurt setup was already correct: `hurtPose` uses `Hurt.png` in `AssetLoader.js`.

The only change made is to show the hurt pose for the full invulnerability period. Before, it only showed during knockback (`knockTimer`); it now also shows during `invulnTimer`, so `Hurt.png` stays visible for the whole invincibility time.

Ensure `assets/Hurt.png` exists and matches the sprite sheet format (128×128 frames, row 0, at least 2 frames). If it has a different layout, adjust the `hurtPose` config in `AssetLoader.js` to match.

User

when it walks it should be using the walk and run pngs

Cursor

Adding a walk animation and switching between walk and run based on movement speed.

Two updates are in place:

1. **AssetLoader.js**
- Added a `walk` animation using `assets/Walk.png` (128×128, 6 frames, slower `frameDelay`).

2. **PlayerEntity.js**
- Movement now switches between three states based on horizontal speed:
  - **idle** – speed < 0.01
  - **walk** – 0.01 ≤ speed < 70% of `MOVE_SPEED` (uses `Walk.png`)
  - **run** – speed ≥ 70% of `MOVE_SPEED` (uses `Run.png`)

Since the player often moves at full speed, you may see mostly the run animation. To favor walk more, either lower the run threshold or add a sprint key and use walk as normal movement and run for sprint.

Ensure `assets/Walk.png` exists and uses 128×128 frames in a row. If the layout differs, change the `walk` config in `AssetLoader.js` accordingly.

User

but when i use arrow keys to move it doesnt use the walk.png

Cursor

Using `walk` for normal movement. Run was shown because the player moves at full `MOVE_SPEED`, so speed was always above the run threshold.

Movement with the arrow keys will now show the walk animation (`Walk.png`). The run animation (`Run.png`) will only play if you add a sprint/run key and update the movement logic accordingly.

User

i only see the idle frame

Cursor

Using a Level-owned tick and more robust frame-count detection to fix frame advancement:

Checking if we need to explicitly call `play()` on the animation:

Ensuring we call `play()` on animations after switching, and verifying the draw/update order.

Adding a fallback to read frame count from our asset definitions when p5play doesn't provide it:

Several changes are in place so the sprite animations advance correctly:

1. **Level.js – frame advancement**
- Switched from global `frameCount` to a Level-owned `_aniTick` to avoid scope/timing issues.
- Frame count is taken from `ani.images?.length` when available.
- If not available, frame count is read from `playerAnis` and `boarAnis`.
- `_aniTick` is reset on level restart.

2. **PlayerEntity.js – animation playback**
- Added an explicit `ani.play()` call after `changeAni()` so animations start playing when switched.

Check that the game runs as expected. If animations still don’t advance, it may be due to the p5play CDN version or how it handles per-animation sprite sheets.

User

none of the sprites are working, use the pngs as sprite sheets

---

Cursor

Updates are in place so all PNGs are loaded and played as sprite sheets via p5play’s `loadAni` + `addAni` flow:

1. **Player** (`PlayerEntity.js`)
- Switched from `addAnis(anis)` to `addAni` for each animation.
- Each PNG (Idle, Walk, Run, Jump, Attack, Hurt, Dead) is passed to `loadAni(spriteSheet, atlas)` and then added with `addAni(name, ani)`.

2. **Boar** (`BoarSystem.js`)
- Same approach: each boar animation (run, throwPose, death) uses `loadAni(def.spriteSheet, atlas)` and `addAni(name, ani)`.
- Removed `cutFrames` and shared sprite sheet handling.

3. **Leaf / Slime** (`TileBuilder.js`, `Leaf.js`)
- Uses `loadAni(leafImg, { w: 64, h: 64, row: 0, frames: 6 })` and `addAni("idle", leafAni)`.

4. **Fire** (`TileBuilder.js`, `Fire.js`)
- Uses `loadAni(fireImg, { w: 32, h: 32, row: 0, frames: 16 })` and `addAni("burn", fireAni)`.

`loadAni` loads each PNG as a sprite sheet and cuts frames with the given `w`, `h`, `row`, and `frames` atlas, so p5play can play them correctly. The existing `_advanceSpriteAnimations` in Level.js still runs as a fallback for frame advancement.

User

now move the sprite up so that its not walking inside the ground

---

Cursor

Updated the player animation offset from -48 to -64 pixels so the sprite sits higher above the ground. It now uses `ANI_OFFSET_Y`, which you can change in `tuning.json` via `player.aniOffsetY` if you want it higher or lower.

User

move it higher by like 2 cm

Cursor

Raised the sprite by about 24 pixels (roughly 2 cm on screen) by changing the default offset from -64 to -88.

User

that didnt seem to do anything

Cursor

Checking whether the offset is applied to each Ani and whether the sprite has an offset property:

Applying the offset to each animation and checking for a sprite-level offset.

Using `sprite.image.offset` — the docs say it offsets the image. For animated sprites, we'll try `sprite.image.offset` or applying the offset each frame.

Several changes are in place so the offset is actually applied:

1. **Tuning source**: `tuning.player.anisOffset.y` is now read correctly (it was using `aniOffsetY`, which didn't exist).

2. **Tuning value**: `tuning.json` has `anisOffset.y = -88` for the player (was `-8`).

3. **Offset in `loadAni` atlas**: Each animation's atlas now includes `offset: { x: 0, y: this.ANI_OFFSET_Y }` when creating animations.

4. **Per-animation offset**: `_setAni` sets `sprite.ani.offset.y` whenever the animation changes, so new animations keep the offset.

5. **Other offset paths**: `anis.offset`, `sprite.image.offset`, and per-ani offsets are still set where relevant.

Try this version; the offset should take effect. If it doesn't, try larger values in `tuning.json` (e.g. `-120`, `-150`) to confirm that offsets are being used.

Level appearance using tileset

*Exported on 3/13/2026 at 20:39:36 EDT from Cursor (2.6.18)*

User

use the tileset png in assets to change the look of the level

---

Cursor

Checking assets and how p5play Tiles use images:

Switching AssetLoader to load `assets/Tileset.png` and extract tiles. Assuming a 32×32 tile grid (common for CraftPix platformer tilesets) and extracting each tile region:

The level tiles are now drawn from `assets/Tileset.png` instead of individual files. Changes in `AssetLoader.js`:

1. **Load the tileset** – `assets/Tileset.png` is loaded once.
2. **Extract tile regions** – Regions are taken from a 32×32 grid with `img.get()`.
3. **Tile mapping** – Regions are assigned to each tile type:
  - Ground (g): (0, 0)

- Deep ground (d): (1, 0)
- Platform left (L): (2, 0)
- Platform right (R): (3, 0)
- Wall left ([): (4, 0)
- Wall right (]): (5, 0)

If a tile type looks wrong, edit the `tileAt(col, row)` calls in `AssetLoader.js`. For example, if ground looks wrong, try other positions like `tileAt(1, 1)` or `tileAt(0, 2)` to match your tileset layout.

You can verify it by opening `http://localhost:8080` (with a server running) and checking that tiles now use the Tileset.png look.

User

i like the tile you used but you need to repeat it because there's a lot of places that are blank

---

Cursor

Using the tile at (0,0) for all tile types so blank spots are filled.

Update complete: every tile type (ground, deep ground, platforms, walls) now uses the same tile from (0, 0), so those blank areas should be filled consistently.

User

i cannot see any tiles

Cursor

Investigating why tiles are invisible: checking tile extraction and trying alternative tile sizes and positions.

Confirming tileset is 544×384 — 32×32 yields 17×12. (0,0) may be empty/transparent. Trying `copy()` for extraction and sampling from multiple positions to find visible tiles.

Two changes were made:

1. **Use `copy()` instead of `get()`** – `createImage()` + `copy()` is more reliable for extracting regions for sprites.
2. **Change base tile to (1, 0)** – (0, 0) is often empty in tilesets; (1, 0) is more likely to be a visible tile.

Refresh and test again. If tiles still don't appear, try other positions such as `(2, 0)`, `(0, 1)`, or `(2, 1)`.

User

use the sound effects in the assets folder for damage, attack, collecting slime, taking damage, and put the looping music in the background

---

Cursor

Starting music on first user interaction (browsers block autoplay):

Here's what was implemented:

SoundManager.js
- `loadAsync()` – loads sounds with optional `loop: true` for music

- `playMusic()` - starts looping music, avoids restarting if already playing

main.js
- Loads from the assets folder:
 - `collectSound.mp3` - collect
 - `damageSound.mp3` - damage
 - `attackSound.mp3` - attack
 - `musicSound.mp3` - background music (loop)
- Starts music on first user click/key to satisfy browser autoplay rules

Game.js
- `leaf:collected` → `collect` (collecting slime)
- `player:damaged` → `damage` (taking damage)
- `player:attacked` → `attack` (player attack)
- `boar:damaged` → `attack` (hitting enemy)
- Music starts when the level loads and restarts on level restart

The game also has `jumpingSound.mp3`; that’s not wired yet. I can add a jump sound next if you want.

User

add jump sound too

Cursor

Jump sound is set up: `jumpingSound.mp3` is loaded and played on the `player:jumped` event.

User

move the sounds into a folder in asset names sounds, move sprites (th epngs) into a folder in assets named images

---

Cursor

Here’s what changed:

Folders:
- `assets/sounds/` – all 5 mp3 files (collectSound, damageSound, attackSound, jumpingSound, musicSound)
- `assets/images/` – all PNG files (Tileset, player sprites, backgrounds, etc.)

Path updates:
- `AssetLoader.js` – image paths use `assets/images/`
- `main.js` – sound paths use `assets/sounds/`
- `data/levels.json` – parallax image paths use `assets/images/`

User

make the platforms look better by using the tiles better

Cursor

Updated the tile mapping so each platform part uses its own tileset region:

- **ground (g)** – flat grassy center (2, 0)
- **deep ground (d)** – darker rock base (0, 1)
- **platform left (L)** – left edge/cap (0, 0)
- **platform right (R)** – right edge/cap (1, 0)
- **wall left ([)** – left wall (3, 0)
- **wall right (])** – right wall (4, 0)

Platforms should now use distinct edges and fills instead of one repeated tile. If a region is wrong or empty, we can tweak the grid coordinates to match your tileset layout.

```
**User**

some places are empty

---

**Cursor**


Update complete. All tile types now use the tile at (1,0) so no blanks show up.
```